

Graphs - Shortest Paths

Fundamental Algorithms

Rodica Potolea, Camelia Lemnaru and Ciprian Oprea

Technical University of Cluj-Napoca
Computer Science Department

Agenda

- 1 Single Source Shortest Paths
 - Dijkstra's Algorithm
 - The Bellman-Ford Algorithm
 - SSSP in DAGs

- 2 All-Pairs Shortest Paths
 - The Floyd-Warshall Algorithm
 - Transitive closure



Agenda

- 1 Single Source Shortest Paths
 - Dijkstra's Algorithm
 - The Bellman-Ford Algorithm
 - SSSP in DAGs
- 2 All-Pairs Shortest Paths
 - The Floyd-Warshall Algorithm
 - Transitive closure



Shortest Paths problem definition

- we are given a weighted graph $G = (V, E)$, $w : E \rightarrow \mathbb{R}$
- a **path** p is a sequence of vertices connected by edges
 $p = \langle v_0, v_1, \dots, v_k \rangle$
- the **weight of a path** $w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$
- the **shortest-path weight** from u to v

$$\delta(u, v) = \begin{cases} \min\{w(p)\} & \text{if } \exists p = \langle u, \dots, v \rangle \\ \infty & \text{otherwise} \end{cases}$$

- a **shortest path** from u to v is any path p with $w(p) = \delta(u, v)$



Optimal substructure of a shortest path

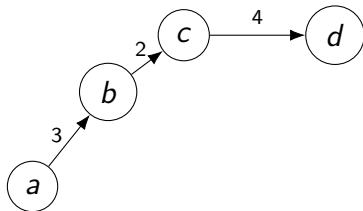
Lemma

Given a weighted graph $G = (V, E)$ with $w : E \rightarrow \mathbb{R}$, let

$p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from v_0 to v_k .

For any i, j , $0 \leq i \leq j \leq k$, the subpath $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ is a shortest path from v_i to v_j .

- if a path is a shortest path, all subpaths are shortest paths



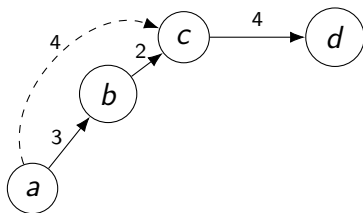


Optimal substructure of a shortest path

Lemma

Given a weighted graph $G = (V, E)$ with $w : E \rightarrow \mathbb{R}$, let $p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from v_0 to v_k . For any i, j , $0 \leq i \leq j \leq k$, the subpath $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ is a shortest path from v_i to v_j .

- if a path is a shortest path, all subpaths are shortest paths



If a shorter subpath existed, the path $v_0 \rightsquigarrow v_k$ would use it.



Optimal substructure of a shortest path

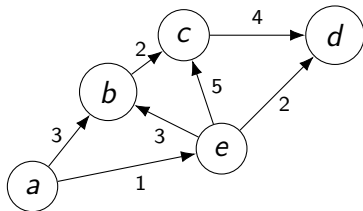
Lemma

Given a weighted graph $G = (V, E)$ with $w : E \rightarrow \mathbb{R}$, let

$p = \langle v_0, v_1, \dots, v_k \rangle$ be a shortest path from v_0 to v_k .

For any i, j , $0 \leq i \leq j \leq k$, the subpath $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ is a shortest path from v_i to v_j .

- if a path is a shortest path, all subpaths are shortest paths
- the reverse is not true



$\langle a, b \rangle$, $\langle b, c \rangle$, $\langle c, d \rangle$,
 $\langle a, b, c \rangle$ and $\langle b, c, d \rangle$ are all
 shortest paths, but $\langle a, b, c, d \rangle$ is not.



Representing shortest paths

Add attributes d and π for each node v .

- $v.d$ - weight of the shortest path from source s to v
- $v.\pi$ - the parent/predecessor in the shortest path from s to v

INITIALIZE-SINGLE-SOURCE(G, s)

```
1  for each vertex  $v \in G.V$ 
2       $v.d = \infty$ 
3       $v.\pi = \text{NIL}$ 
4   $s.d = 0$ 
```



Relaxation technique

- try to improve the shortest path found so far
- can we find a shorter path to v by passing through u ?

RELAX(u, v, w)

```
1  if  $v.d > u.d + w(u, v)$   
2       $v.d = u.d + w(u, v)$   
3       $v.\pi = u$ 
```



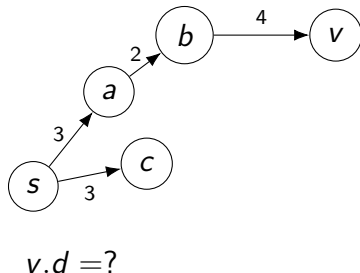
Relaxation technique

- try to improve the shortest path found so far
- can we find a shorter path to v by passing through u ?

RELAX(u, v, w)

```

1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
  
```





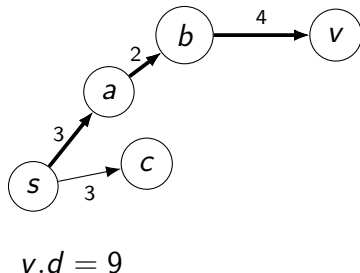
Relaxation technique

- try to improve the shortest path found so far
- can we find a shorter path to v by passing through u ?

RELAX(u, v, w)

```

1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
  
```





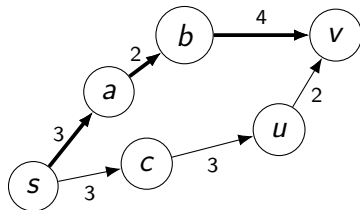
Relaxation technique

- try to improve the shortest path found so far
- can we find a shorter path to v by passing through u ?

RELAX(u, v, w)

```

1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
  
```



$v.d = 9$ $u.d = 6$



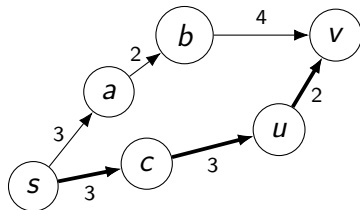
Relaxation technique

- try to improve the shortest path found so far
- can we find a shorter path to v by passing through u ?

RELAX(u, v, w)

```

1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
  
```



$$\begin{aligned}
 &\cancel{v.d = 9} \quad u.d = 6 \\
 &v.d = 6 + 2 = 8
 \end{aligned}$$



Dijkstra's Algorithm

DIJKSTRA(G, w, s)

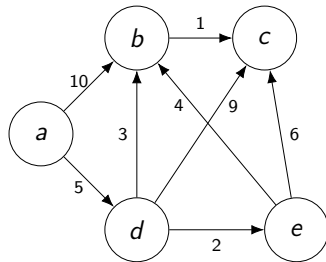
```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
```



Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
```



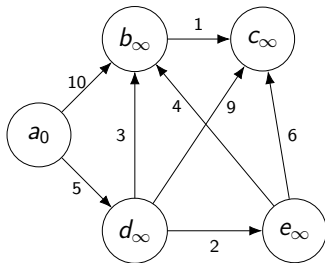


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d
a	NIL	0
b	NIL	∞
c	NIL	∞
d	NIL	∞
e	NIL	∞



Dijkstra's Algorithm

DIJKSTRA(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 $S = \emptyset$

3 $Q = G.V$

4 **while** $Q \neq \emptyset$

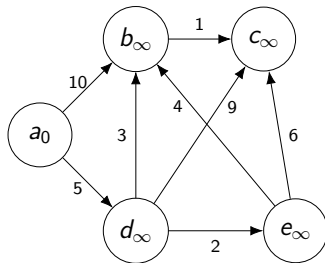
5 $u = \text{EXTRACT-MIN}(Q)$

6 $S = S \cup \{u\}$

7 **for** each vertex $v \in G.Adj[u]$

8 RELAX(u, v, w)

(a)



	π	d
a	NIL	0
b	NIL	∞
c	NIL	∞
d	NIL	∞
e	NIL	∞

} Q



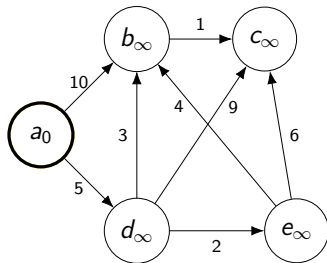
Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```

(a)



	π	d	
a	NIL	0	} S
b	NIL	∞	
c	NIL	∞	} Q
d	NIL	∞	
e	NIL	∞	



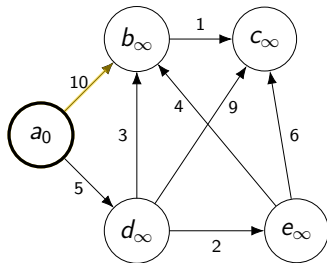
Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```

(a)



	π	d	
a	NIL	0	} S
b	NIL	∞	
c	NIL	∞	} Q
d	NIL	∞	
e	NIL	∞	

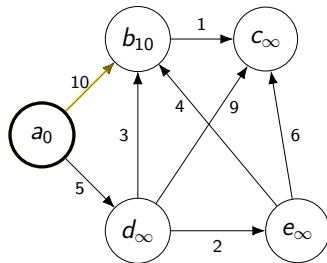
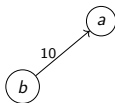


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
b	a	10	
c	NIL	∞	} Q
d	NIL	∞	
e	NIL	∞	

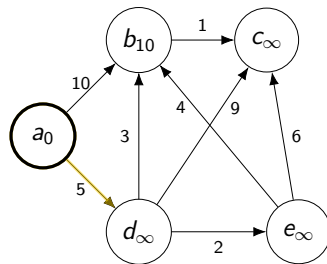
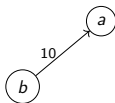


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
b	a	10	
c	NIL	∞	} Q
d	NIL	∞	
e	NIL	∞	

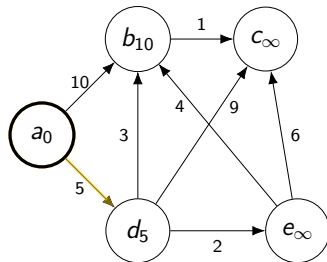
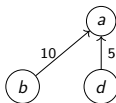


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
b	a	10	} Q
c	NIL	∞	
e	NIL	∞	

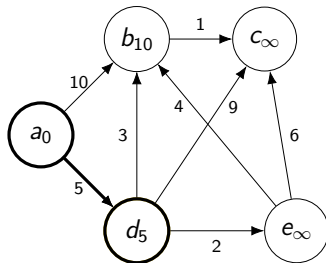
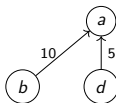


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
b	a	10	
c	NIL	∞	} Q
e	NIL	∞	

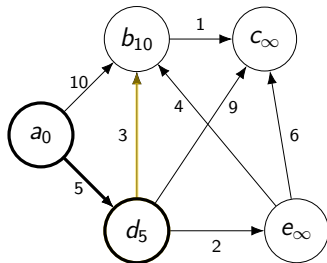
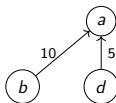


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
b	a	10	} Q
c	NIL	∞	
e	NIL	∞	

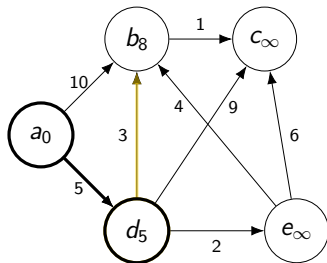
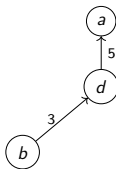


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
b	d	8	} Q
c	NIL	∞	
e	NIL	∞	

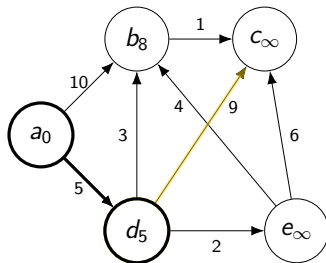
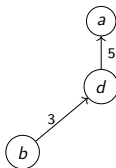


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
b	d	8	} Q
c	NIL	∞	
e	NIL	∞	

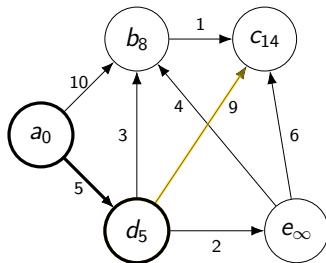
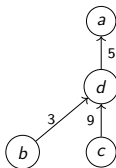


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
b	d	8	} Q
c	d	14	
e	NIL	∞	

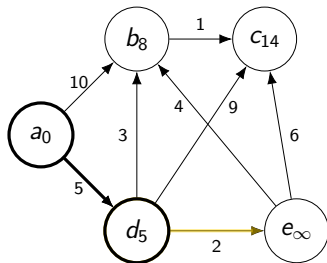
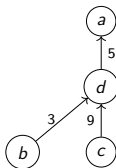


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
b	d	8	} Q
c	d	14	
e	NIL	∞	

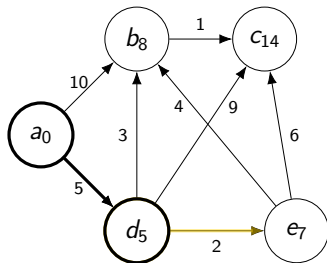
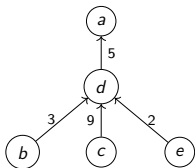


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	} Q
b	d	8	
c	d	14	

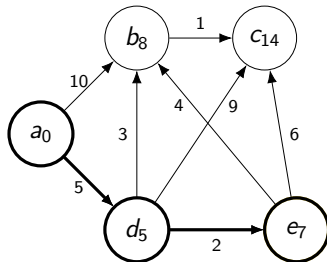
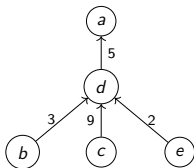


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	} Q
c	d	14	

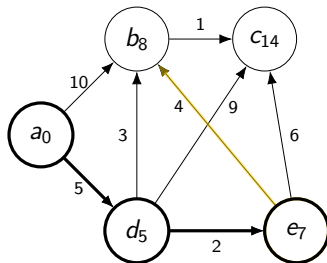
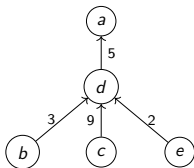


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	} Q
c	d	14	

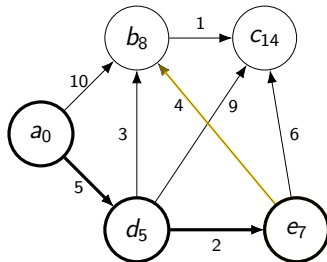
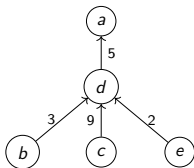


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	} Q
c	d	14	

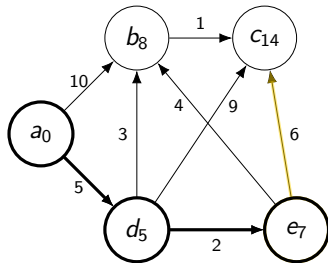
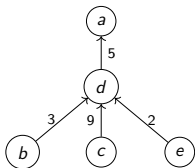


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	} Q
c	d	14	

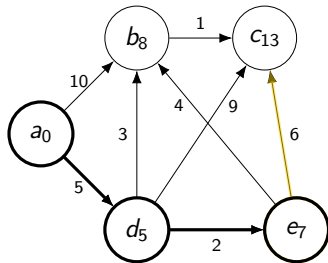
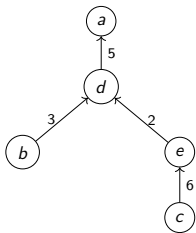


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	} Q
c	e	13	

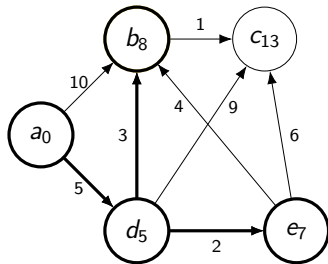
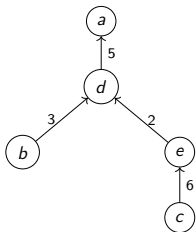


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	
c	e	13	} Q

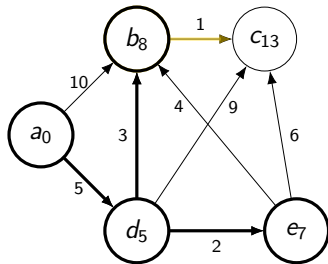
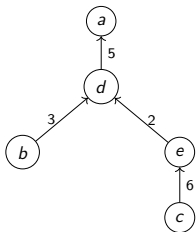


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	
			} Q
c	e	13	

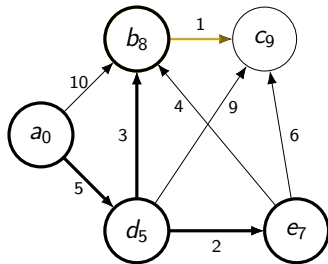
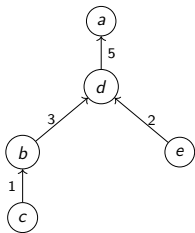


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	
c	b	9	} Q

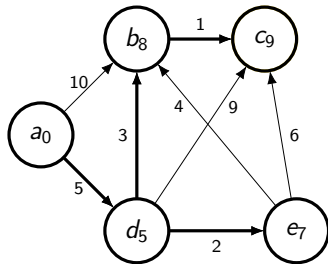
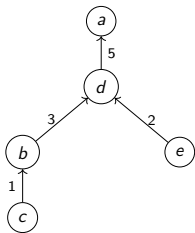


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	} S
a	NIL	0	
d	a	5	
e	d	7	
b	d	8	
c	b	9	

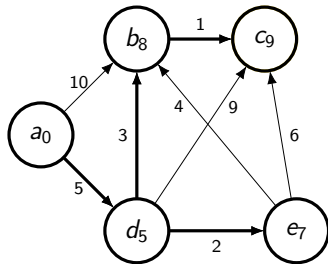
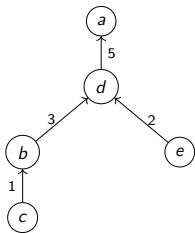


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	
a	NIL	0	} S
d	a	5	
e	d	7	
b	d	8	
c	b	9	

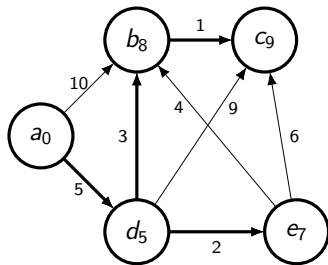
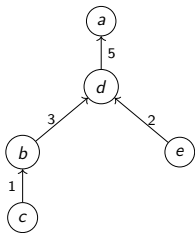


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d	} S
a	NIL	0	
d	a	5	
e	d	7	
b	d	8	
c	b	9	

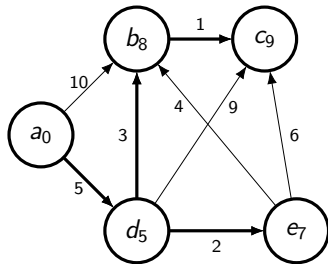
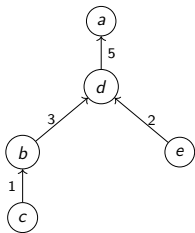


Dijkstra's Algorithm

DIJKSTRA(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```



	π	d
a	NIL	0
d	a	5
e	d	7
b	d	8
c	b	9

} S



Dijkstra's Algorithm - Complexity

DIJKSTRA(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
```



Dijkstra's Algorithm - Complexity

DIJKSTRA(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
```

regular array

binary heap

Fibonacci heap



Dijkstra's Algorithm - Complexity

DIJKSTRA(G, w, s)

1 INITIALIZE-SINGLE-SOURCE(G, s)

2 $S = \emptyset$

3 $Q = G.V$

4 **while** $Q \neq \emptyset$

5 $u = \text{EXTRACT-MIN}(Q)$

6 $S = S \cup \{u\}$

7 **for** each vertex $v \in G.Adj[u]$

8 RELAX(u, v, w)

$\nearrow O(V)$

regular array

binary heap

Fibonacci heap



Dijkstra's Algorithm - Complexity

```

DIJKSTRA( $G, w, s$ )
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for each vertex  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
  
```

Complexity analysis annotations:

- Line 1: $O(V)$
- Line 3: $O(V)$
- Line 4: $O(V)$
- Line 5: $O(V)$
- Line 6: $O(V)$
- Line 7: $O(V)$
- Line 8: $O(V)$

Overall complexity: $O(V^2)$ (regular array), $O(V \log V)$ (binary heap), $O(V \log V)$ (Fibonacci heap).

regular array

binary heap

Fibonacci heap



Dijkstra's Algorithm - Complexity

DIJKSTRA(G, w, s)
 1 INITIALIZE-SINGLE-SOURCE(G, s) $\rightarrow O(V)$
 2 $S = \emptyset$
 3 $Q = G.V$ $\rightarrow O(V)/O(V)/O(V)$
 4 **while** $Q \neq \emptyset$ $\rightarrow |V|$ steps
 5 $u = \text{EXTRACT-MIN}(Q)$
 6 $S = S \cup \{u\}$
 7 **for** each vertex $v \in G.Adj[u]$
 8 RELAX(u, v, w)

regular array

binary heap

Fibonacci heap



Dijkstra's Algorithm - Complexity

DIJKSTRA(G, w, s)
 1 INITIALIZE-SINGLE-SOURCE(G, s) $\rightarrow O(V)$
 2 $S = \emptyset$
 3 $Q = G.V$ $\rightarrow O(V)/O(V)/O(V)$
 4 **while** $Q \neq \emptyset$ $\rightarrow |V|$ steps
 5 $u = \text{EXTRACT-MIN}(Q)$ $\rightarrow O(V)/O(\log V)/O(\log V)$
 6 $S = S \cup \{u\}$
 7 **for** each vertex $v \in G.Adj[u]$
 8 RELAX(u, v, w)

regular array

binary heap

Fibonacci heap



Dijkstra's Algorithm - Complexity

$$\text{DIJKSTRA}(G, w, s)$$
1 INITIALIZE-SINGLE-SOURCE(G, s)
$$2 \quad S = \emptyset$$
$$3 \quad Q = G.V$$
4 **while** $Q \neq \emptyset$ 5 $u = \text{EXTRACT-MIN}(Q)$
$$6 \quad S = S \cup \{u\}$$

```

7   for each vertex  $v \in G.Adj[u]$ 

```

8 RELAX(u, v, w) $O(V)$ $O(V) / O(V) / O(V)$ $|V|$ steps
$$O(V) / O(\log V) / O(\log V)$$

each neighbor
for every vertex,
so $|E|$ steps in total

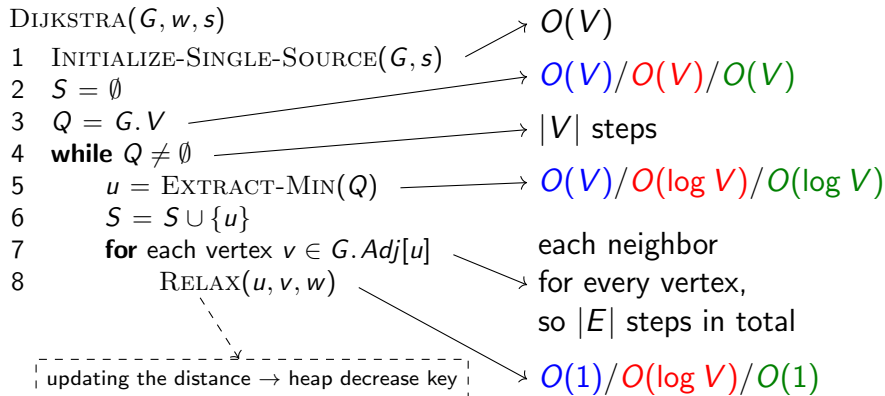
regular array

binary heap

Fibonacci heap



Dijkstra's Algorithm - Complexity



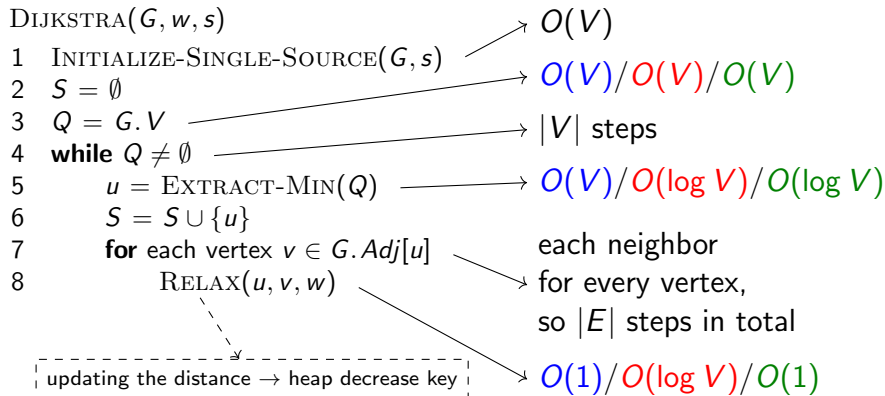
regular array

binary heap

Fibonacci heap



Dijkstra's Algorithm - Complexity



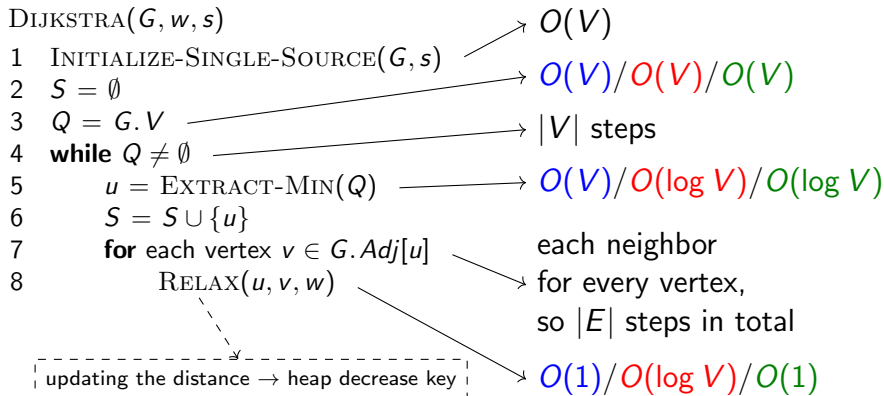
regular array
 $O(V \cdot V + E \cdot 1)$
 $= O(V^2)$

binary heap

Fibonacci heap



Dijkstra's Algorithm - Complexity



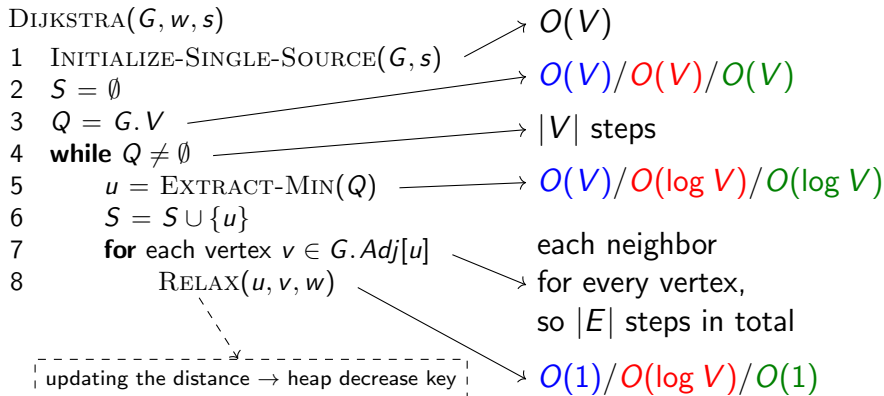
regular array
 $O(V \cdot V + E \cdot 1)$
 $= O(V^2)$

binary heap
 $O(V \log V + E \log V)$
 $= O(E \log V)$

Fibonacci heap



Dijkstra's Algorithm - Complexity



regular array
 $O(V \cdot V + E \cdot 1)$
 $= O(V^2)$

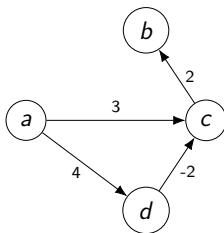
binary heap
 $O(V \log V + E \log V)$
 $= O(E \log V)$

Fibonacci heap
 $O(V \log V + E)$



Negative-weight edges

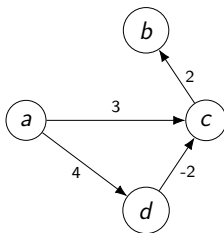
- Dijkstra's algorithm doesn't work if we have negative edges



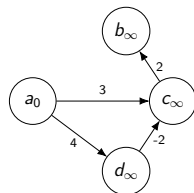


Negative-weight edges

- Dijkstra's algorithm doesn't work if we have negative edges



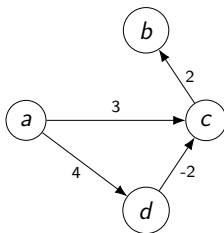
Dijkstra's solution



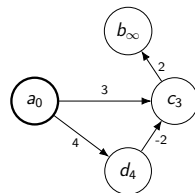


Negative-weight edges

- Dijkstra's algorithm doesn't work if we have negative edges



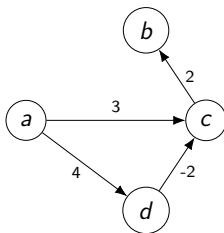
Dijkstra's solution



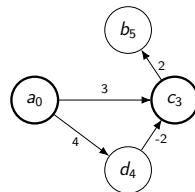


Negative-weight edges

- Dijkstra's algorithm doesn't work if we have negative edges



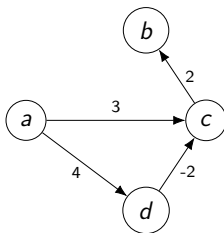
Dijkstra's solution



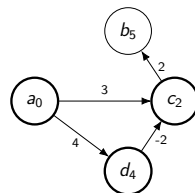


Negative-weight edges

- Dijkstra's algorithm doesn't work if we have negative edges



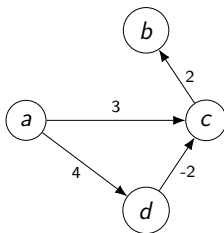
Dijkstra's solution



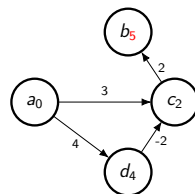


Negative-weight edges

- Dijkstra's algorithm doesn't work if we have negative edges



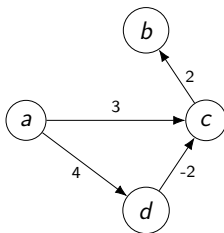
Dijkstra's solution



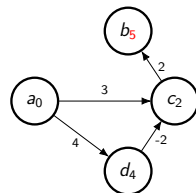


Negative-weight edges

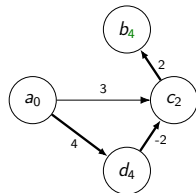
- Dijkstra's algorithm doesn't work if we have negative edges



Dijkstra's solution

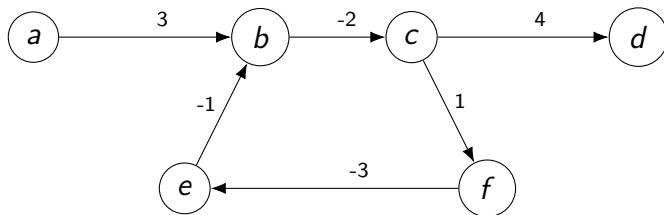


Correct solution





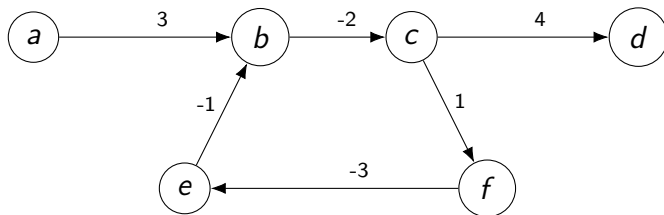
Negative-weight cycles



- what is the shortest path from a to d ?



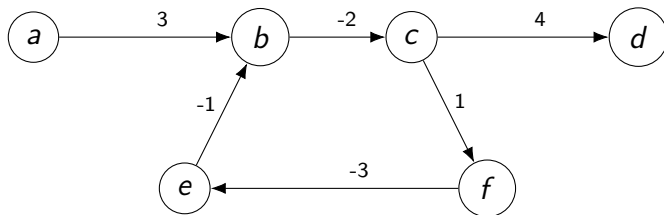
Negative-weight cycles



- what is the shortest path from a to d ?
 - $w(\langle a, b, c, d \rangle) = 5$



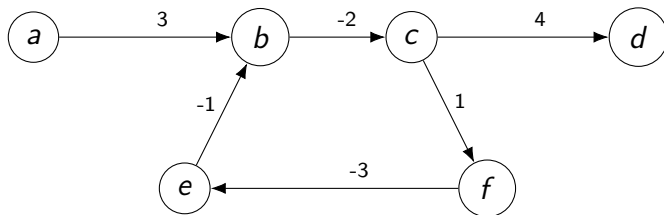
Negative-weight cycles



- what is the shortest path from a to d ?
 - $w(\langle a, b, c, d \rangle) = 5$
 - $w(\langle a, b, c, f, e, b, c, d \rangle) = 0$



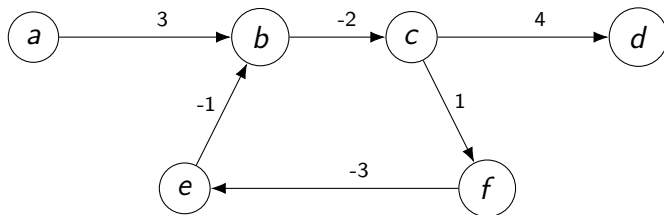
Negative-weight cycles



- what is the shortest path from a to d ?
 - $w(\langle a, b, c, d \rangle) = 5$
 - $w(\langle a, b, c, f, e, b, c, d \rangle) = 0$
 - $w(\langle a, b, c, f, e, b, c, f, e, b, c, f, e, b, c, d \rangle) = -5$



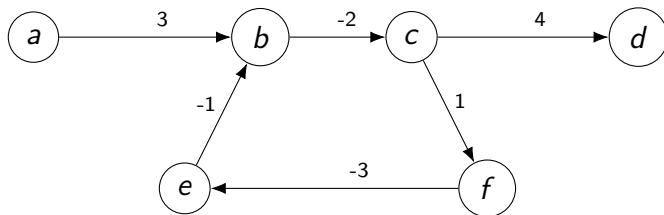
Negative-weight cycles



- what is the shortest path from a to d ?
 - $w(\langle a, b, c, d \rangle) = 5$
 - $w(\langle a, b, c, f, e, b, c, d \rangle) = 0$
 - $w(\langle a, b, c, f, e, b, c, f, e, b, c, f, e, b, c, d \rangle) = -5$
 - ...
 - $-\infty$



Negative-weight cycles



- what is the shortest path from a to d ?
 - $w(\langle a, b, c, d \rangle) = 5$
 - $w(\langle a, b, c, f, e, b, c, d \rangle) = 0$
 - $w(\langle a, b, c, f, e, b, c, f, e, b, c, f, e, b, c, d \rangle) = -5$
 - ...
 - $-\infty$
- the shortest path is not defined if we have negative-weight cycles



Bellman-Ford - approach

- if we have negative edges, some edges need to be “*relaxed*” more than once



Bellman-Ford - approach

- if we have negative edges, some edges need to be “*relaxed*” more than once
- perform $|V| - 1$ steps
 - call RELAX on every edge



Bellman-Ford - approach

- if we have negative edges, some edges need to be “*relaxed*” more than once
- perform $|V| - 1$ steps
 - call RELAX on every edge
- check for negative-weight cycles



Bellman-Ford - approach

- if we have negative edges, some edges need to be “*relaxed*” more than once
- perform $|V| - 1$ steps
 - call RELAX on every edge
- check for negative-weight cycles
 - if an edge can be “*relaxed*” even further it means that we have a negative-weight cycle



The Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```

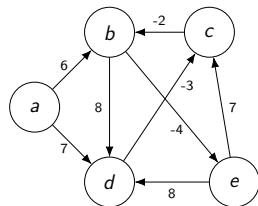
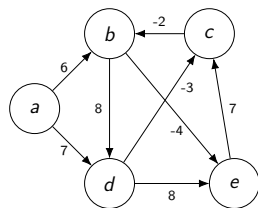


The Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
  
```



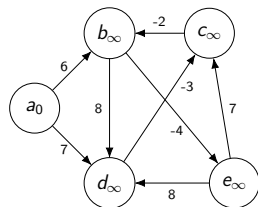
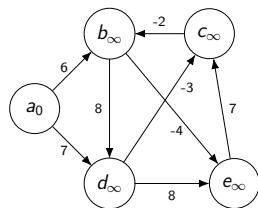


The Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
  
```



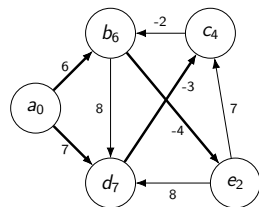
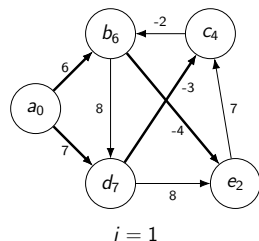


The Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
  
```



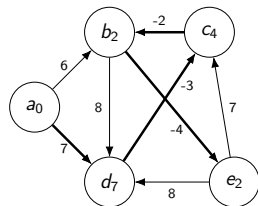
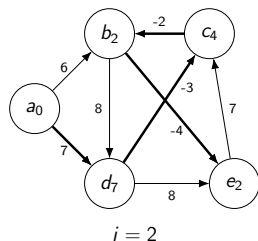


The Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
  
```



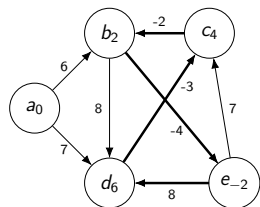
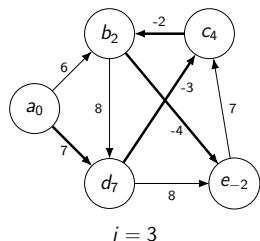


The Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
  
```



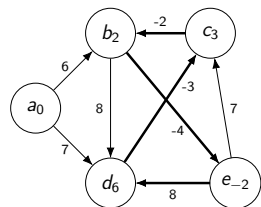
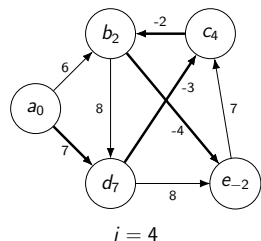


The Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
  
```



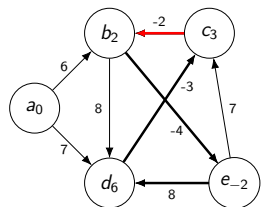
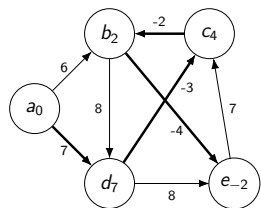


The Bellman-Ford Algorithm

BELLMAN-FORD(G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
  
```





The Bellman-Ford Algorithm - Complexity

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```



The Bellman-Ford Algorithm - Complexity

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$   $\longrightarrow O(V)$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```



The Bellman-Ford Algorithm - Complexity

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$   $\longrightarrow O(V)$ 
3      for each edge  $(u, v) \in G.E$   $\longrightarrow \times O(E)$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```



The Bellman-Ford Algorithm - Complexity

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```

Complexity annotations:

- Line 2: $\rightarrow O(V)$
- Line 3: $\rightarrow \times O(E)$
- Lines 5-7: $\rightarrow O(E)$



The Bellman-Ford Algorithm - Complexity

BELLMAN-FORD(G, w, s)

```
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for each edge  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for each edge  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE
```

Annotations for complexity analysis:

- Line 2: $\rightarrow O(V)$
- Line 3: $\rightarrow \times O(E)$
- Lines 5-7: $\rightarrow O(E)$

Complexity: $O(V \cdot E)$



SSSP in DAGs - approach

- DAG = Directed Acyclic Graph
- no cycles $\rightarrow$ no negative-weight cycles
- the vertices can be topologically sorted
 - if $\exists u \rightsquigarrow v$, then u comes before v



SSSP in DAGs - approach

- DAG = Directed Acyclic Graph
- no cycles $\rightarrow$ no negative-weight cycles
- the vertices can be topologically sorted
 - if $\exists u \rightsquigarrow v$, then u comes before v
- a single pass over the vertices in topologically sorted order
 - relax each edge that leaves the vertex



SSSP in DAGs Algorithm

DAG-SHORTEST-PATHS(G, w, s)

- 1 topologically sort the vertices of G // see seminar
- 2 INITIALIZE-SINGLE-SOURCE(G, s)
- 3 **for** each vertex u , taken in topologically sorted order
- 4 **for** each vertex $v \in G.Adj[u]$
- 5 RELAX(u, v, w)



SSSP in DAGs Algorithm - Complexity

DAG-SHORTEST-PATHS(G, w, s)

- 1 topologically sort the vertices of G // see seminar
- 2 INITIALIZE-SINGLE-SOURCE(G, s)
- 3 **for** each vertex u , taken in topologically sorted order
- 4 **for** each vertex $v \in G.Adj[u]$
- 5 RELAX(u, v, w)



SSSP in DAGs Algorithm - Complexity

DAG-SHORTEST-PATHS(G, w, s)

- 1 topologically sort the vertices of G // see seminar
- 2 INITIALIZE-SINGLE-SOURCE(G, s)
- 3 **for** each vertex u , taken in topologically sorted order
- 4 **for** each vertex $v \in G.Adj[u]$
- 5 RELAX(u, v, w)

→ $O(V + E)$



SSSP in DAGs Algorithm - Complexity

DAG-SHORTEST-PATHS(G, w, s)

```
1  topologically sort the vertices of  $G$  // see seminar  $\rightarrow O(V + E)$ 
2  INITIALIZE-SINGLE-SOURCE( $G, s$ )  $\rightarrow O(V)$ 
3  for each vertex  $u$ , taken in topologically sorted order
4      for each vertex  $v \in G.Adj[u]$ 
5          RELAX( $u, v, w$ )
```



SSSP in DAGs Algorithm - Complexity

DAG-SHORTEST-PATHS(G, w, s)

- 1 topologically sort the vertices of G // see seminar $\rightarrow O(V + E)$
- 2 INITIALIZE-SINGLE-SOURCE(G, s) $\rightarrow O(V)$
- 3 **for** each vertex u , taken in topologically sorted order
- 4 **for** each vertex $v \in G.Adj[u]$ $\rightarrow |V|$ steps
- 5 RELAX(u, v, w)



SSSP in DAGs Algorithm - Complexity

DAG-SHORTEST-PATHS(G, w, s)

- 1 topologically sort the vertices of G // see seminar $\rightarrow O(V + E)$
- 2 INITIALIZE-SINGLE-SOURCE(G, s) $\rightarrow O(V)$
- 3 **for** each vertex u , taken in topologically sorted order $\rightarrow |V|$ steps
- 4 **for** each vertex $v \in G.Adj[u]$ $\rightarrow$ each neighbor
- 5 RELAX(u, v, w) $\rightarrow$ for every vertex, so $|E|$ steps in total



SSSP in DAGs Algorithm - Complexity

DAG-SHORTEST-PATHS(G, w, s)

```

1  topologically sort the vertices of  $G$  // see seminar
2  INITIALIZE-SINGLE-SOURCE( $G, s$ )
3  for each vertex  $u$ , taken in topologically sorted order
4      for each vertex  $v \in G.Adj[u]$ 
5          RELAX( $u, v, w$ )
  
```

$\rightarrow O(V + E)$
 $\rightarrow O(V)$
 $\rightarrow |V|$ steps
 each neighbor
 for every vertex,
 so $|E|$ steps in total

Complexity: $O(V + E + V + E) = O(V + E)$



Agenda

- 1 Single Source Shortest Paths
 - Dijkstra's Algorithm
 - The Bellman-Ford Algorithm
 - SSSP in DAGs
- 2 All-Pairs Shortest Paths
 - The Floyd-Warshall Algorithm
 - Transitive closure



All-Pairs Shortest Paths

- compute the shortest path between every pair of vertices
 - the result is a matrix
- we assume that the vertices are numbered $1, 2, \dots, n$
- the graph is given as an adjacency matrix with weights $W = (w_{ij})$

$$w_{ij} = \begin{cases} 0 & \text{if } i = j \\ \text{the weight of the edge } (i, j) & \text{if } i \neq j \text{ and } (i, j) \in E \\ \infty & \text{if } i \neq j \text{ and } (i, j) \notin E \end{cases}$$

- simple approach: use SSSP on every node



Floyd-Warshall - Approach

- denote by $d_{ij}^{(k)}$ the weight of a shortest path from i to j for which all intermediary nodes are in the set $\{1, 2, \dots, k\}$

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0 \\ \min \left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \right) & \text{if } k \geq 1 \end{cases}$$

- dynamic programming
- $D^{(n)} = \left(d_{ij}^{(n)} \right)$ gives the final answer - $d_{ij}^{(n)} = \delta(i, j)$



The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```
1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
```

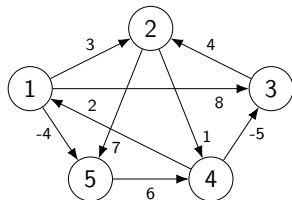


The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```





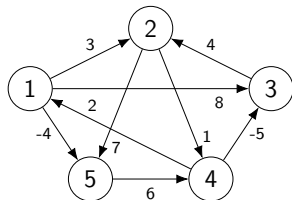
The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 

```



$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

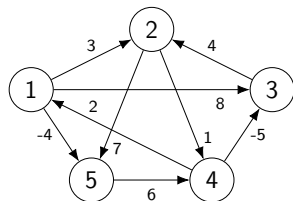


The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```



$k = 1$

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

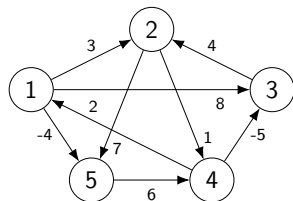


The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```



$k = 2$

$$D^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

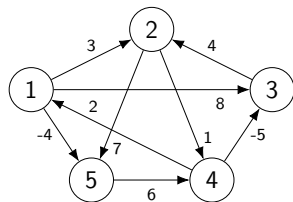


The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```



$k = 3$

$$D^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

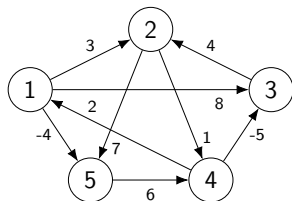


The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```



$k = 4$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

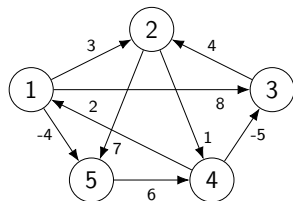


The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```



$k = 5$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

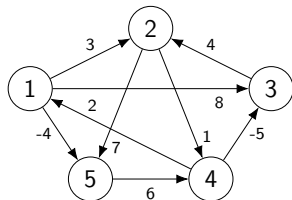


The Floyd-Warshall Algorithm

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```



$$D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$



Floyd-Warshall - Time Complexity

FLOYD-WARSHALL(W)

```
1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min (d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
```



Floyd-Warshall - Time Complexity

FLOYD-WARSHALL(W)

```
1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix  $\rightarrow O(n)$ 
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
```



Floyd-Warshall - Time Complexity

FLOYD-WARSHALL(W)

1 $n = W.rows$

2 $D^{(0)} = W$

3 **for** $k = 1$ **to** n

4 let $D^{(k)} = (d_{ij}^{(k)})$ be a new $n \times n$ matrix $\rightarrow O(n)$

5 **for** $i = 1$ **to** n $\rightarrow \times O(n)$

6 **for** $j = 1$ **to** n

7 $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$

8 **return** $D^{(n)}$



Floyd-Warshall - Time Complexity

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix  $\rightarrow O(n)$ 
5      for  $i = 1$  to  $n$   $\rightarrow \times O(n)$ 
6          for  $j = 1$  to  $n$   $\rightarrow \times O(n)$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```



Floyd-Warshall - Time Complexity

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix  $\rightarrow O(n)$ 
5      for  $i = 1$  to  $n$   $\rightarrow \times O(n)$ 
6          for  $j = 1$  to  $n$   $\rightarrow \times O(n)$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 
  
```

Time Complexity: $O(n^3)$ or $O(V^3)$



Floyd-Warshall - Space Complexity

FLOYD-WARSHALL(W)

```
1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4      let  $D^{(k)} = \left(d_{ij}^{(k)}\right)$  be a new  $n \times n$  matrix
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min \left( d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \right)$ 
8  return  $D^{(n)}$ 
```



Floyd-Warshall - Space Complexity

FLOYD-WARSHALL(W)

1 $n = W.rows$

2 $D^{(0)} = W$

3 **for** $k = 1$ **to** n

4 let $D^{(k)} = (d_{ij}^{(k)})$ be a new $n \times n$ matrix

5 **for** $i = 1$ **to** n

6 **for** $j = 1$ **to** n

7 $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$

8 **return** $D^{(n)}$

naive approach:

a new matrix for each k ,
so $O(n \times n^2) = O(n^3)$



Floyd-Warshall - Space Complexity

FLOYD-WARSHALL(W)

1 $n = W.rows$

2 $D^{(0)} = W$

3 **for** $k = 1$ **to** n

4 let $D^{(k)} = (d_{ij}^{(k)})$ be a new $n \times n$ matrix

5 **for** $i = 1$ **to** n

6 **for** $j = 1$ **to** n

7 $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$

8 **return** $D^{(n)}$

naive approach:

a new matrix for each k ,
so $O(n \times n^2) = O(n^3)$

actually, at step k
we only need the matrix
from step $k - 1$



Floyd-Warshall - Space Complexity

FLOYD-WARSHALL(W)

1 $n = W.rows$

2 $D^{(0)} = W$

3 **for** $k = 1$ **to** n

4 let $D^{(k)} = (d_{ij}^{(k)})$ be a new $n \times n$ matrix

5 **for** $i = 1$ **to** n

6 **for** $j = 1$ **to** n

7 $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$

8 **return** $D^{(n)}$

naive approach:

a new matrix for each k ,
so $O(n \times n^2) = O(n^3)$

actually, at step k
we only need the matrix
from step $k - 1$

in fact, we can work
on the same matrix



Floyd-Warshall - Space Complexity

FLOYD-WARSHALL(W)

1 $n = W.rows$

2 $D^{(0)} = W$

3 **for** $k = 1$ **to** n

4 let $D^{(k)} = (d_{ij}^{(k)})$ be a new $n \times n$ matrix

5 **for** $i = 1$ **to** n

6 **for** $j = 1$ **to** n

7 $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$

8 **return** $D^{(n)}$

naive approach:

a new matrix for each k ,
so $O(n \times n^2) = O(n^3)$

actually, at step k
we only need the matrix
from step $k - 1$

in fact, we can work
on the same matrix

Space Complexity: $O(n^2)$ or $O(V^2)$



Floyd-Warshall - Path Construction

- build a matrix $\Pi = (\pi_{ij})$, where π_{ij} is the parent of node j in the shortest path from i to j

$$\pi_{ij}^{(0)} = \begin{cases} \text{NIL} & \text{if } i = j \text{ or } w_{ij} = \infty \\ i & \text{if } i \neq j \text{ and } w_{ij} < \infty \end{cases}$$

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{if } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{if } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$

- $\Pi = \Pi^{(n)}$



The Floyd-Warshall Algorithm with Path (1)

FLOYD-WARSHALL-2(W)

```
1   $n = W.rows$ 
2  for  $i = 1$  to  $n$ 
3      for  $j = 1$  to  $n$ 
4           $d_{ij} = w_{ij}$ 
5          if  $i == j$  or  $w_{ij} == \infty$ 
6               $\pi_{ij} = NIL$ 
7          else  $\pi_{ij} = i$ 
8  for  $k = 1$  to  $n$ 
9      for  $i = 1$  to  $n$ 
10         for  $j = 1$  to  $n$ 
11             if  $d_{ij} > d_{ik} + d_{kj}$ 
12                  $d_{ij} = d_{ik} + d_{kj}$ 
13                  $\pi_{ij} = \pi_{kj}$ 
14  return  $D, \Pi$ 
```



The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```
1  if  $\pi_{ij} == \text{NIL}$ 
2      return NIL
3   $path = \{j\}$ 
4  while  $j \neq i$ 
5       $j = \pi_{ij}$ 
6       $path = \{j\} \cup path$ 
7  return  $path$ 
```

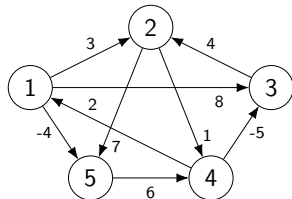


The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2      return NIL
3   $path = \{j\}$ 
4  while  $j \neq i$ 
5       $j = \pi_{ij}$ 
6       $path = \{j\} \cup path$ 
7  return  $path$ 
  
```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$$i = 2; j = 5$$



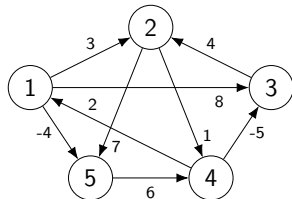
The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2      return NIL
3   $\text{path} = \{j\}$ 
4  while  $j \neq i$ 
5       $j = \pi_{ij}$ 
6       $\text{path} = \{j\} \cup \text{path}$ 
7  return  $\text{path}$ 

```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$$i = 2; j = 5$$

$$\text{path} = \{ 5 \}$$

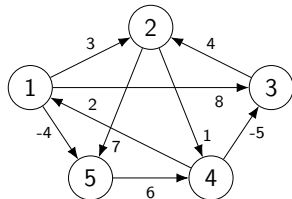


The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2      return NIL
3  path = {j}
4  while  $j \neq i$ 
5       $j = \pi_{ij}$ 
6      path = {j}  $\cup$  path
7  return path
  
```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$$i = 2; j = \cancel{1} \\ \text{path} = \{ \quad 5 \}$$

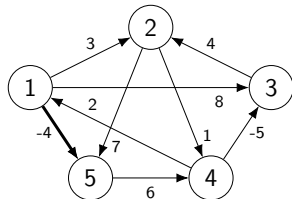


The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2    return NIL
3  path = {j}
4  while  $j \neq i$ 
5     $j = \pi_{ij}$ 
6    path = {j}  $\cup$  path
7  return path
  
```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$$i = 2; j = \cancel{1}$$

$$\text{path} = \{ 1, 5 \}$$

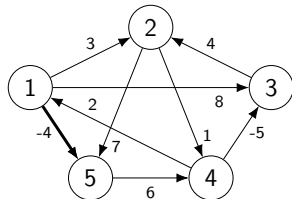


The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2      return NIL
3  path = {j}
4  while  $j \neq i$ 
5       $j = \pi_{ij}$ 
6      path = {j}  $\cup$  path
7  return path
  
```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$i = 2; j = \cancel{5} \cancel{1} 4$
 path = { 1, 5 }

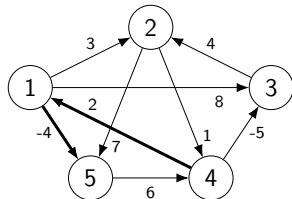


The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2      return NIL
3  path = {j}
4  while  $j \neq i$ 
5       $j = \pi_{ij}$ 
6      path = {j}  $\cup$  path
7  return path
  
```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$i = 2; j = 5 \neq 4$
 $path = \{ 4, 1, 5 \}$



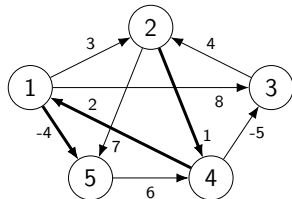
The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2      return NIL
3  path = {j}
4  while  $j \neq i$ 
5       $j = \pi_{ij}$ 
6      path = {j}  $\cup$  path
7  return path

```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$i = 2; j = \cancel{5} \cancel{1} \cancel{4} 2$
 $path = \{ 4, 1, 5 \}$

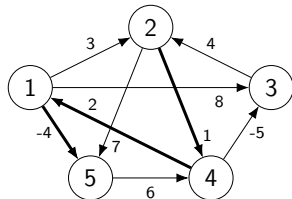


The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2      return NIL
3  path = {j}
4  while  $j \neq i$ 
5       $j = \pi_{ij}$ 
6      path = {j}  $\cup$  path
7  return path
  
```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$i = 2; j = 5$
~~1~~ ~~4~~ ~~2~~
 path = { 2, 4, 1, 5 }

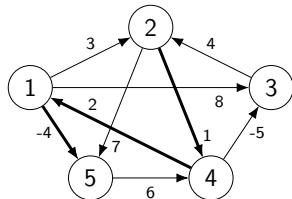


The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2    return NIL
3  path = {j}
4  while  $j \neq i$ 
5     $j = \pi_{ij}$ 
6    path = {j}  $\cup$  path
7  return path
  
```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$i = 2; j = 5$
~~1~~ ~~4~~ ~~2~~
 path = { 2, 4, 1, 5 }

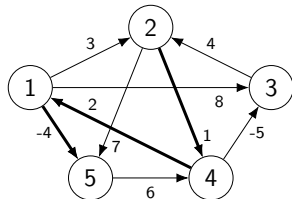


The Floyd-Warshall Algorithm with Path (2)

PATH-CONSTRUCTION(Π, i, j)

```

1  if  $\pi_{ij} == \text{NIL}$ 
2    return NIL
3  path = {j}
4  while  $j \neq i$ 
5     $j = \pi_{ij}$ 
6    path = {j}  $\cup$  path
7  return path
  
```



$$\Pi = \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}$$

$i = 2; j = 5$
~~1~~ ~~4~~ ~~2~~
 path = { 2, 4, 1, 5 }



Transitive closure

Definition

Given a directed graph $G = (V, E)$, with $V = \{1, 2, \dots, n\}$, we define the **transitive closure** as $G = (V, E^*)$, where $E^* = \{(i, j) : \exists i \rightsquigarrow j\}$.

- we want to determine whether G contains a path from i to j for all pairs $i, j \in V$
- solution 1: run Floyd-Warshall with $w_{ij} = 1, \forall (i, j) \in E$ and check if $d_{ij} < \infty$
- solution 2: change arithmetical operations into logical operations

$$t_{ij}^{(0)} = \begin{cases} 0 & \text{if } i \neq j \text{ and } (i, j) \notin E \\ 1 & \text{if } i = j \text{ or } (i, j) \in E \end{cases}$$

$$t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee \left(t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)} \right), k \geq 1$$



The transitive closure algorithm

TRANSITIVE-CLOSURE(G)

```
1   $n = |G.V|$ 
2  let  $T = (t_{ij})$  be a new  $n \times n$  matrix
3  for  $i = 1$  to  $n$ 
4      for  $j = 1$  to  $n$ 
5          if  $i == j$  or  $(i, j) \in G.E$ 
6               $t_{ij} = 1$ 
7          else  $t_{ij} = 0$ 
8  for  $k = 1$  to  $n$ 
9      for  $i = 1$  to  $n$ 
10         for  $j = 1$  to  $n$ 
11              $t_{ij} = t_{ij} \vee (t_{ik} \wedge t_{kj})$ 
12  return  $T$ 
```



Bibliography

- Cormen, Thomas H., et al., *"Introduction to algorithms."*, MIT press, 2009, cap. 24, cap. 25